Algorithms

INTRODUCTION & TOPOLOGICAL SORT

Logistics

- All the materials in English
- For administrative questions - send a private message to all instructors in Piazza

Teaching Assistants

- Daniel Prigan - daniel.prigan@post.runi.ac.il
- Yael Berkman - yael.berkman@post.runi.ac.il
- Daniel Karalnik – daniel.Karalnik@post.runi.ac.il
- Liam Tal – liam.tal@post.runi.ac.il

Recitations' instructors

HW support and grade

Reception Hours

- Location (zoom or campus) and time will be announced on Moodle.
- 3 weekly reception hours by the recitations' instructors
- 3 additional reception hours will focus on helping you solve the homework / understand your mistakes.

Homework Assignments

- 8 assignments.
- Coping with the HW (by yourself) is the best way to study in this course.
- You are encouraged to come to TA reception hours, where you will be given assistance and explanations regarding solving the homework.
- To encourage you to do your best and try by yourself, not all homework will be checked, however submission is mandatory.
- Will be published on Wednesday evening; you will have until Sunday evening (week and a half) to submit.
- No extensions.

This Week

- Definitions
- Graph representations
- Directed acyclic graphs (DAG)
- Topological sort

Graph Theory - Definitions

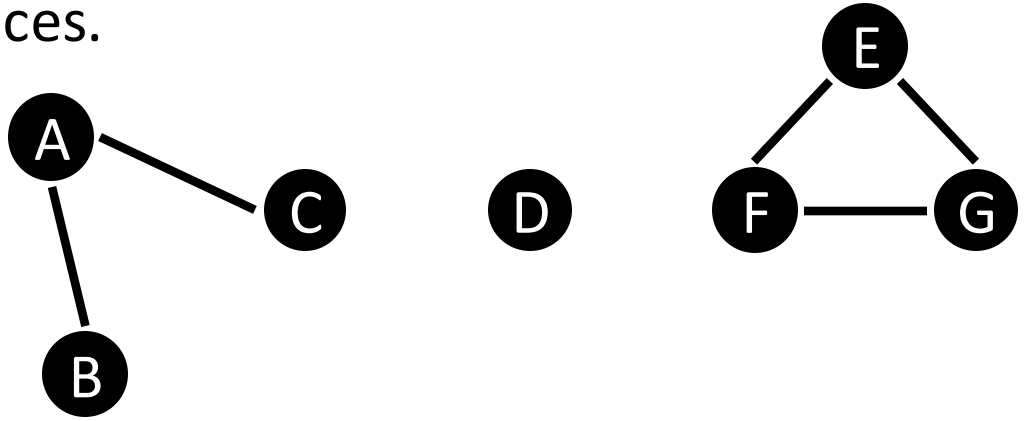
A graph G is defined by two sets:

- V: The set of vertices
- E: The set of edges. Every edge is a subset of 2 elements from V

- Notations: $|V| = n$, $|E| = m$

Undirected Graph

$G(V, E)$ is a graph with the set of vertices V and set of undirected edges E that connect the vertices.

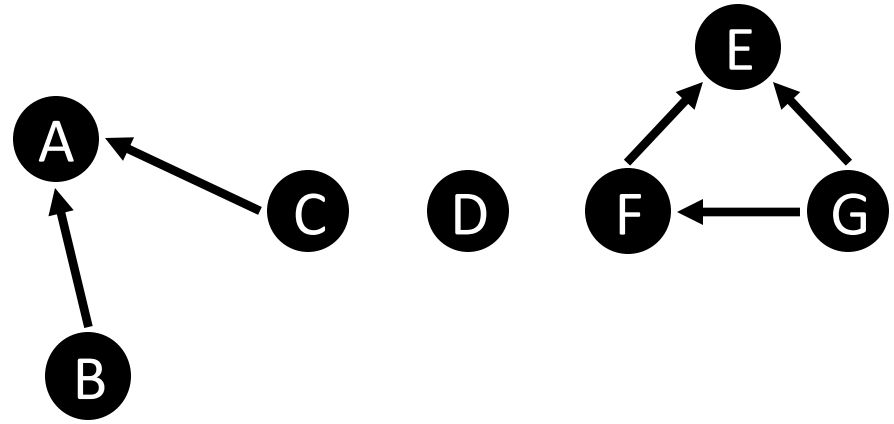


$V = \{A, B, C, D, E, F, G\}$

$E = \{(A, B), (A, C), (E, F), (E, G), (F, G)\}$

Directed Graph

$G(V, E)$ is a graph with the set of vertices V and set the of ordered pairs of vertices E called directed edges



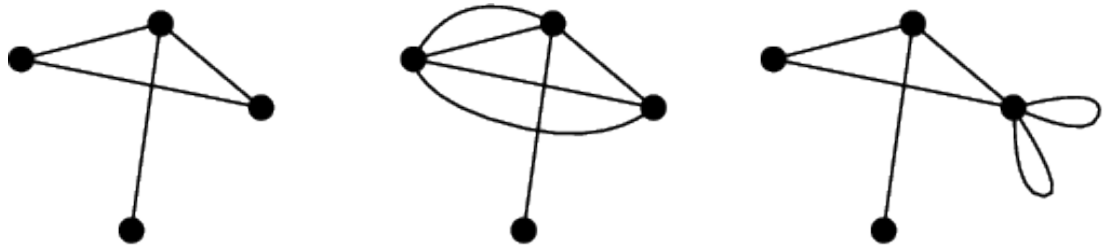
$V = \{A, B, C, D, E, F, G\}$

$E = \{(B, A), (C, A), (F, E), (G, E), (G, F)\}$

Simple Undirected Graph

An unweighted, undirected graph containing no self-loops or multiple (parallel) edges.

A simple graph may be either connected or disconnected



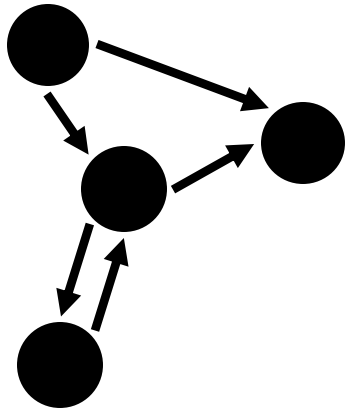
A simple graph

Non-simple graphs

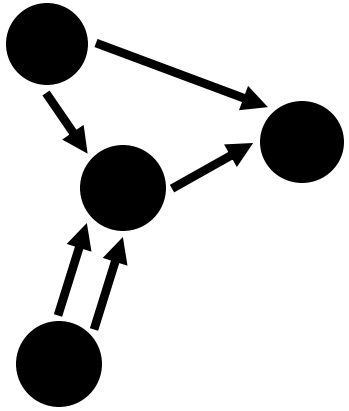
Simple Directed Graph

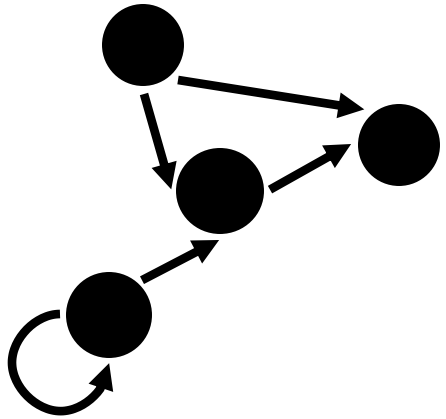
An unweighted, directed graph containing no self-loops or multiple (parallel) edges

Edges in different directions are not considered parallel.



A simple graph

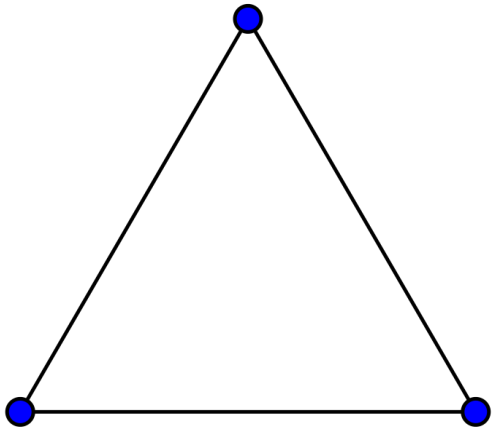


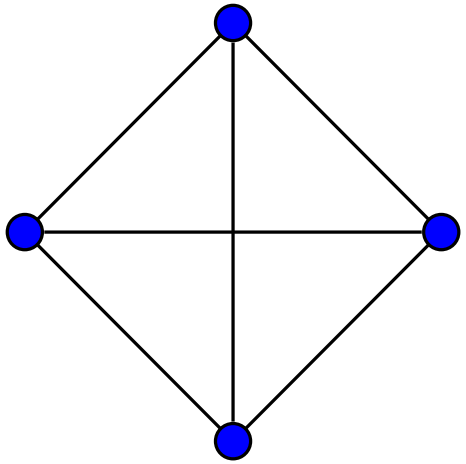


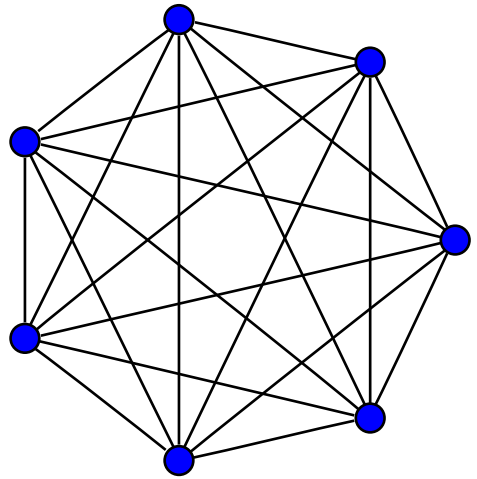
Non-simple graphs

Complete Graph (Clique)

An undirected graph in which every pair of vertices is connected by an edge.



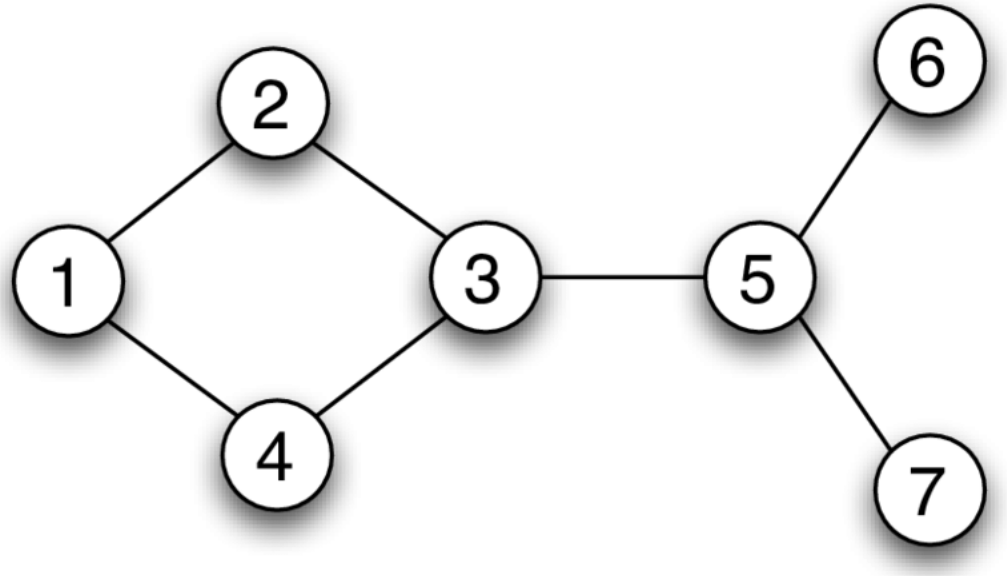




How many edges exist in a clique? $\frac{n(n-1)}{2}$

Path

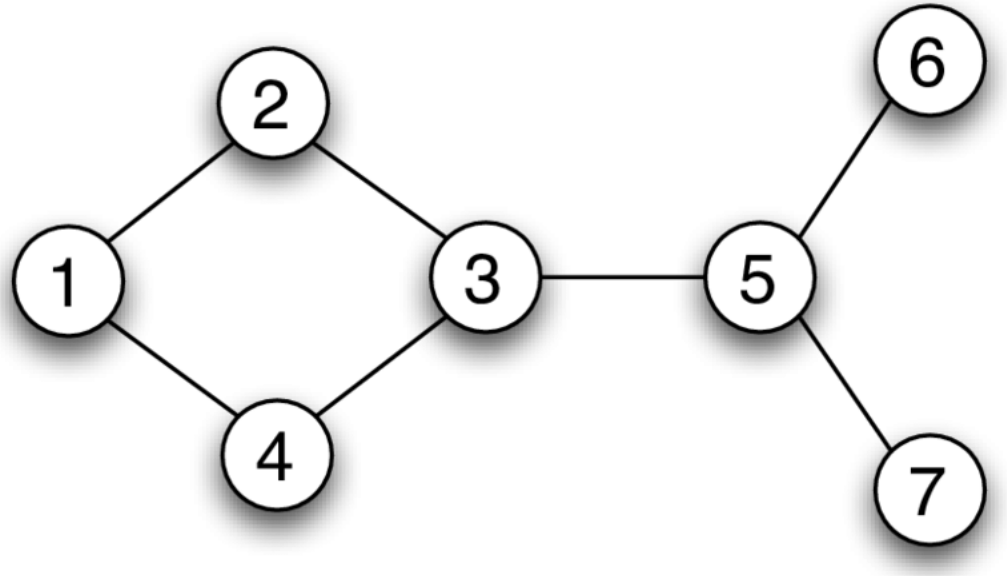
A sequence of edges which connect a sequence of vertices



$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$

Simple Path

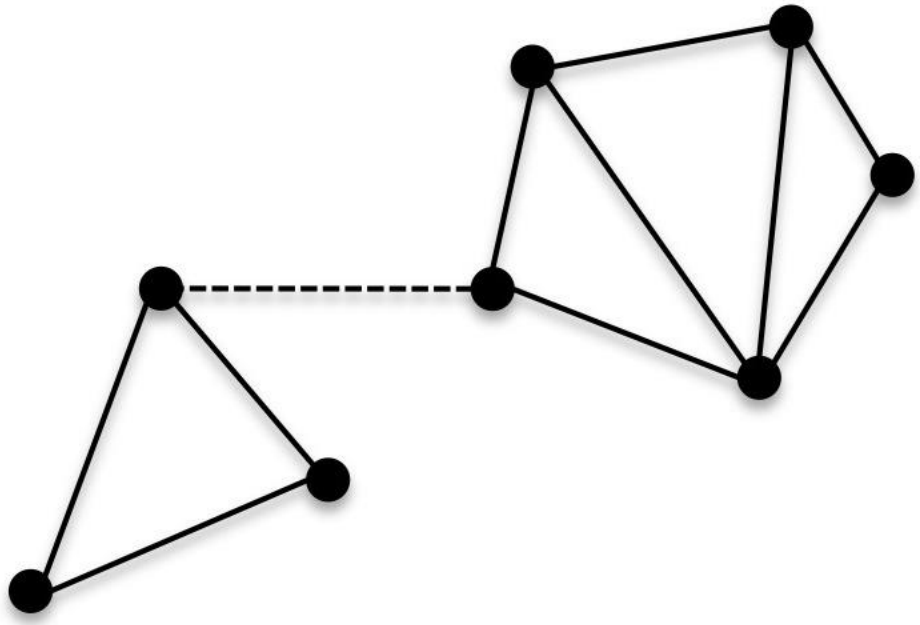
A path with distinct vertices



$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6$

An Undirected Connected Graph

An **undirected** graph that includes at least one path between every pair of vertices

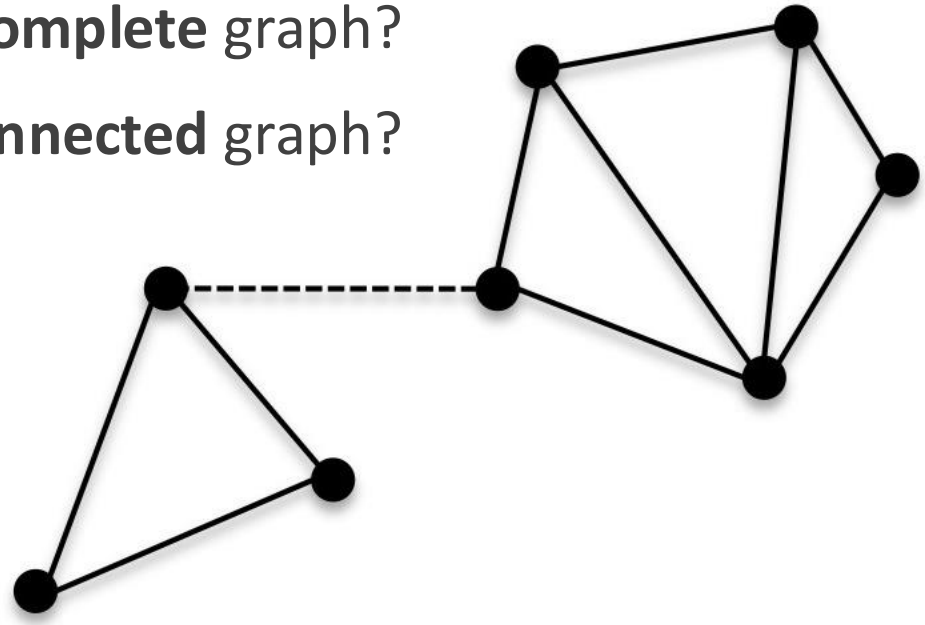


An Undirected Connected Graph

An **undirected** graph with a path between every pair of vertices

Is every **connected** graph a **complete** graph?

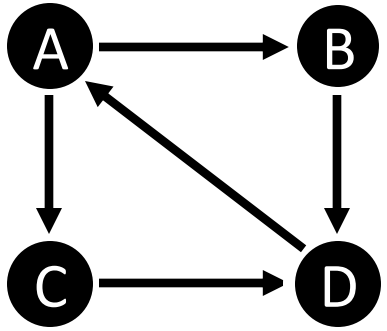
Is every **complete** graph a **connected** graph?



Strongly Connected Directed Graph

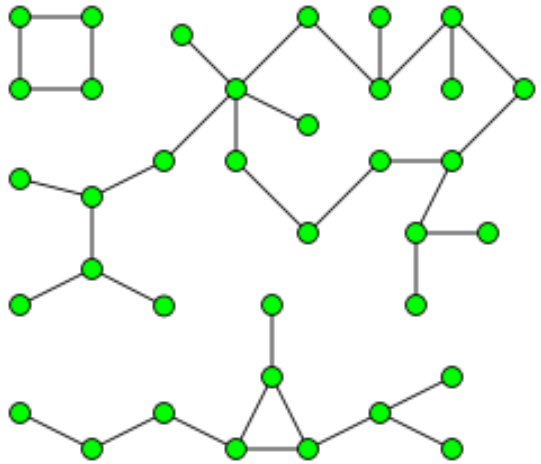
A **directed** graph is strongly connected if for every pair of vertices u and v, there exists a **directed** path from u to v**.**

Note: This definition refers only to directed graphs while the previous definition refers to undirected graphs



Connected Components

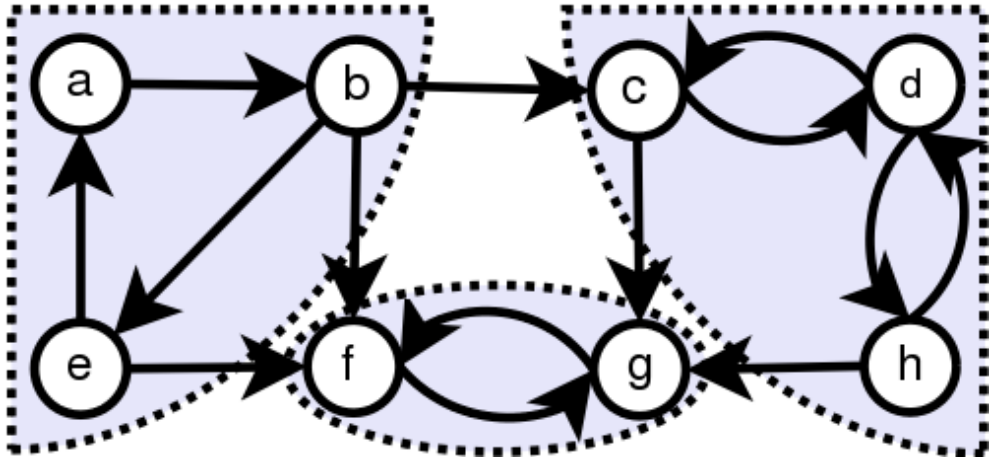
Given an **undirected** graph, a set of vertices is **connected** if and only if there is a path between every pair of vertices in the set. A connected component is a maximal such set. The collection of all connected components of a graph G forms a partition of the vertices of G.



An undirected graph **cannot** have edges between different connected components

Strongly Connected Component

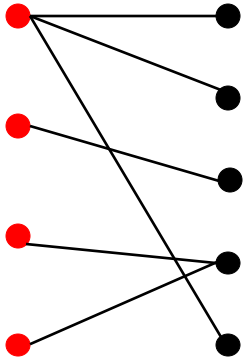
Given a directed graph, a set of vertices is strongly connected if and only if there is a directed path between every pair of vertices in the set. A strongly connected component is a maximal such set. The collection of all strongly connected components of a graph G forms a partition of the vertices of G.

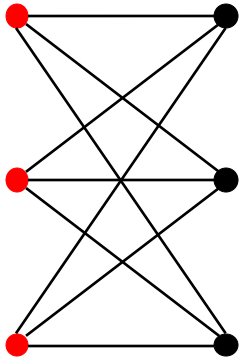


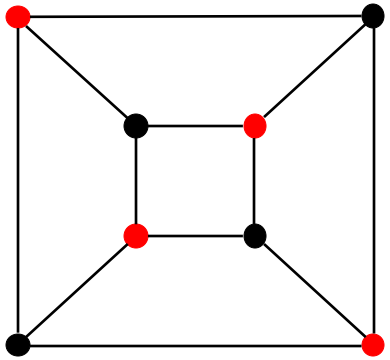
A directed graph **might** have edges between different connected components

Bipartite Graph

A graph with two mutually exclusive sets of vertices and a set of edges such that every edge connects two vertices from different sets. Formally, $G = (V = \{V_1, V_2\}, E \subseteq V_1 \times V_2)$

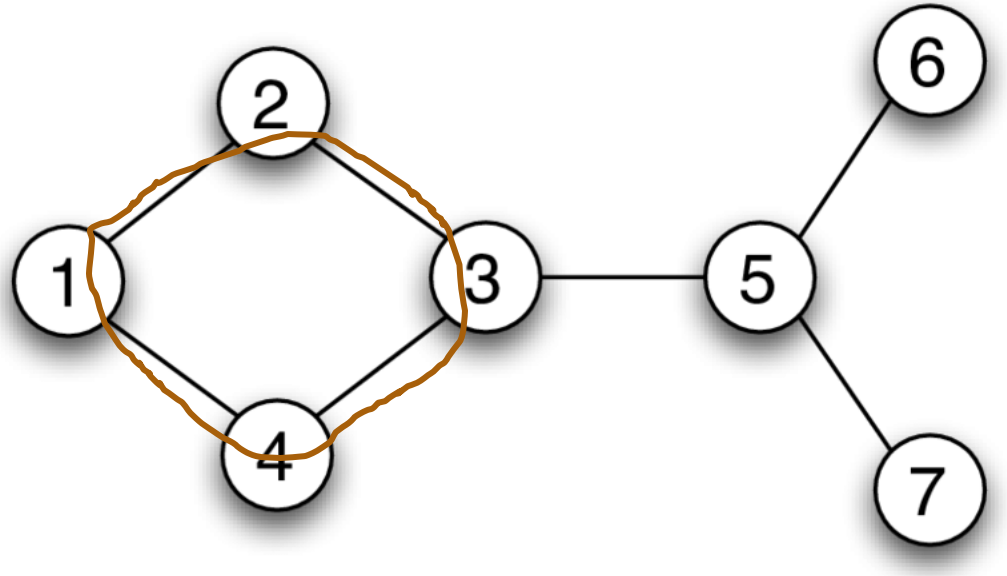






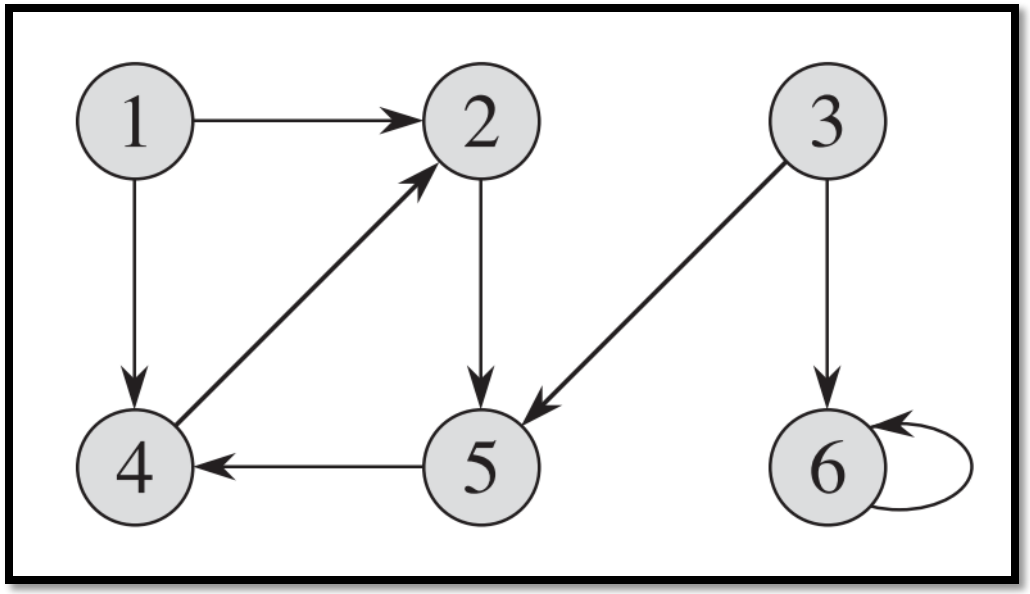
Simple Cycle

A path that starts and ends at the same vertex, containing no repetitions of vertices or edges



$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$

Graph Representations



Adjacency Matrix

If a graph contains no self-loops, the principal diagonal is 0

In an undirected graph, the adjacency matrix is symmetric with respect to the principal diagonal

Checking whether an edge $(u, v) \in E$ can be done in $O(1)$

The size of the representation depends only on the number of vertices - $O(n^2)$

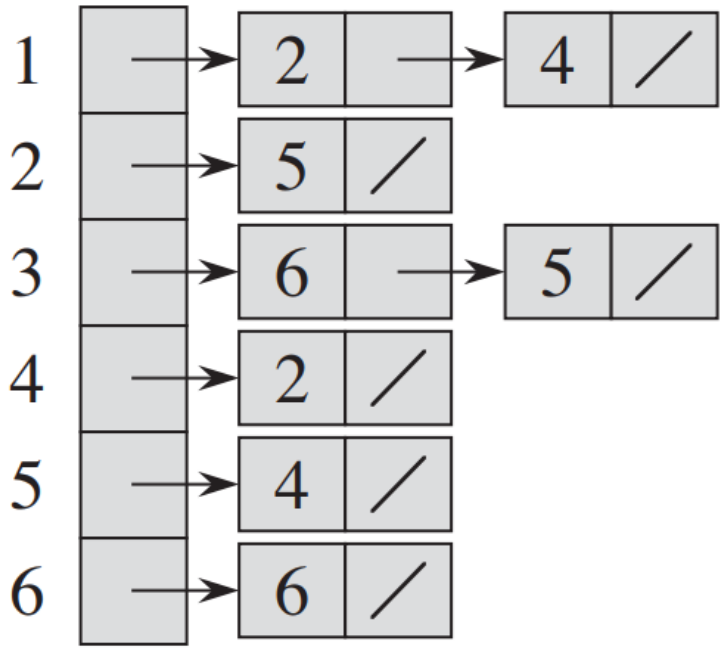
Is this representation good or bad?

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Graph Representation – Adjacency List

Checking whether an edge $(u, v) \in E$ is done by searching a linked-list

The size of the representation is $O(n + m)$



Handshaking Theorem

Definition: The degree of a vertex (denoted $deg(v)$) is the number of edges adjacent to it.

Claim: Let $G = (V, E)$ be an undirected graph. The sum of all degrees in G is twice the number of the edges:

$$\sum_{v \in V} deg(v) = 2 \cdot m$$

Proof:

Each edge is incident to exactly two vertices.

When we add up the degrees of all the vertices, we count each endpoint once, thus, each edge is counted twice. ■

Exercise 1

Claim: Every undirected graph has an even number of vertices of odd degree.

Proof:

By the *Handshaking theorem* the sum of all degrees is $2m$, and thus even.

Split the vertices into two disjoint sets: vertices with even and odd degrees.

The sum of the degrees of vertices with even degree is even.

The sum of the degrees of vertices with odd degree must also be even.

Thus, there must be an even number of vertices of odd degree. ■

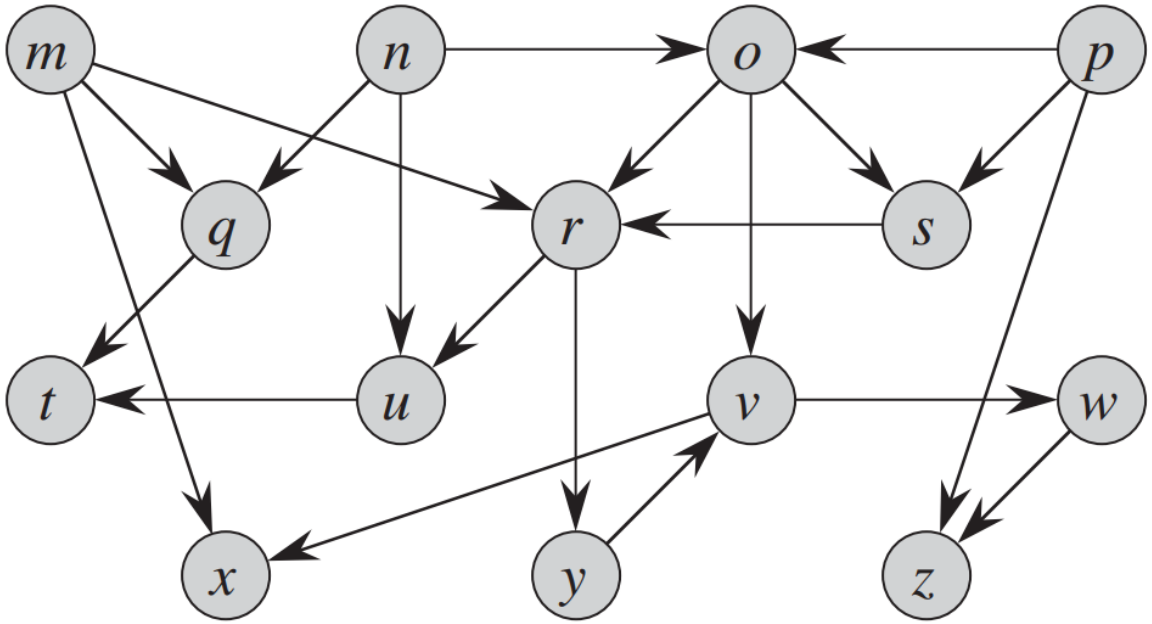
$$2m = \sum_{d \in Even(V)} \deg(v) + \sum_{v \in odd(V)} \deg(v)$$

$$even \quad = \quad even \quad + \quad even$$

Directed Acyclic Graphs (DAGs)

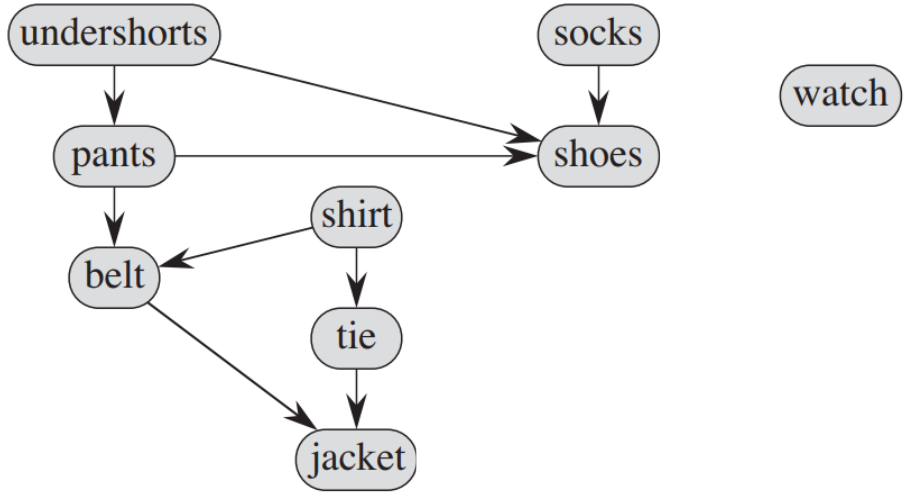
Definition: A directed graph with no directed cycles

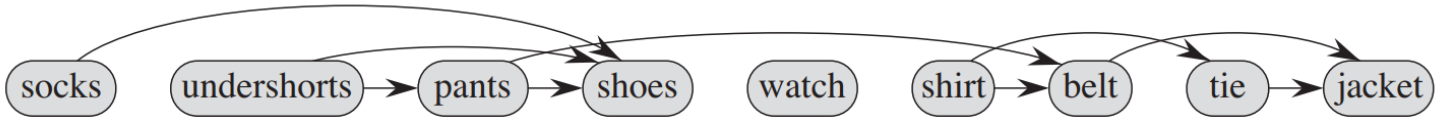
Note: The underlying undirected graph may include cycles (e.g., o-r-s-p-o)



Topological Sort

A topological sort of a graph $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge $(u, v) \in E$, then u appears before v in the ordering.





Topological Sort Algorithm

Algorithm 1 TOPOLOGICAL SORT(G)

```
1: D ← In-degree of all vertices in G
2: Q ← Enqueue all in-degree zero vertices
3: while Q do
4:     Dequeue and output a vertex
5:     Reduce in-degree of all vertices adjacent to it by 1
6:     Enqueue any of these vertices whose in-degree became zero
```

Running time analysis:

1. Initializing D takes $O(n+m)$
2. Initializing Q takes $O(n)$
3. Enqueueing a vertex with in-degree zero and outputting it takes $O(1)$. This is done $O(n)$ times.
4. Reducing the in-degree of all adjacent vertices takes $O(m)$.
5. Total time: $O(n+m)$

Topological Sort - Correctness

Definition: A vertex v is a source if its in-degree is 0.

First, a useful claim.

Claim: Every directed acyclic graph (DAG) has at least one source vertex.

Proof:

Pick some arbitrary vertex v_1. If v_1 is a source, we are done.

Otherwise, let v_2 be a vertex such that $(v_2, v_1) \in E$. $v_2 \neq v_1$ since the graph is a DAG.

If v_2 is a source, we are done. Otherwise, proceed backward (as with v_1).

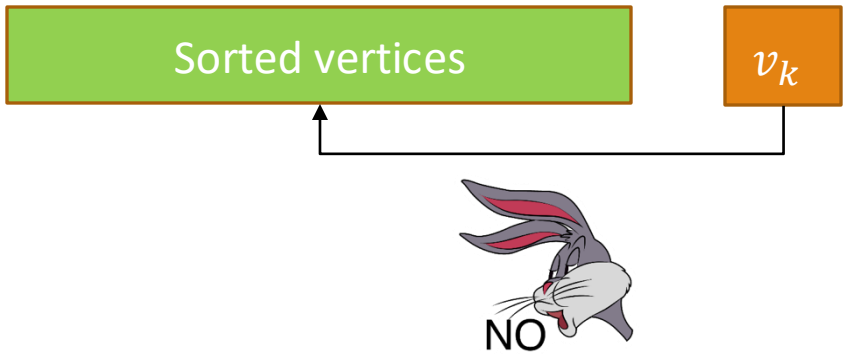
After at most n steps of backward traversals, we will find a source vertex, since the graph is finite and contains no cycles. ■

Topological Sort - Correctness

Claim: The algorithm produces a topological sort of a DAG G.

Proof: By induction. We show that for every k, after k iterations, the output of the algorithm is a valid topological sort of the graph induced by the vertices that are already in the output.

In other words, for the vertex v_k chosen in iteration k, there are no edges from it to previously sorted vertices.



Topological Sort - Correctness

Claim: The algorithm produces a topological sort of a DAG G.

Induction base case: k=0, empty output.

Induction Step:

Consider the k-th iteration of the algorithm. According to the algorithm, the output is extended by a vertex with in-degree zero, (such a vertex exists in every DAG). This vertex is appended to the end of the output, which is, by the induction hypothesis, a valid topological sort of the first k-1 vertices.

Let v_k be the output vertex. Let A be the set of vertices that were output before v_k.

Let $v_i \in A$. For every edge (v_i, v_k), since v_k is appended to the end of the output, the output is valid for such edges.

Also, no edge (v_k, v_i) exists since v_i was output earlier, and in-degree(v_i) was 0 at the time it was added to A. ■

Exercise: DAGs and Topological Sorts

Claim: G has a topological sort if and only if G is a Directed Acyclic Graph (DAG)

Proof:

→ if G has a topological sort then G is a DAG: We assume $\neg B$ and show $\neg A$.

Let G be a directed graph, and let C be a directed cycle in G. We show that G has no topo-sort.

Assume towards contradiction that G has a topological sort.

Let v_i be the lowest indexed node in C and let v_j be the node just before v_i in the cycle.

Therefore, (v_j, v_i) is an edge. By our choice of i, we know that v_i precedes v_j in the topo-sort. Since (v_j, v_i) is an edge, we get a contradiction.



Exercise: DAGs and Topological Sorts

← if G is a DAG then G has a topo-sort:

Base case: A DAG with a single vertex has topo-sort (the vertex itself).

I.H: if G is a DAG of size n, then G has a topo-sort.

Induction step: If G is a DAG with $n + 1$ vertices, let s be a source vertex. Remove s and the outgoing edges from s.

The resulting graph is a DAG, since we only removed a vertex and some edges, thus we could have not possibly created a cycle.

By the I.H, we know that the resulting graph (which consists of n vertices) has a topological sort. We append that topo-sort after s to obtain a topo-sort of the graph G. ■

Exercise: DAGs and Topological Sorts

Note: The proof in the previous slide is identical to the correctness proof.

Therefore (an alternative proof):

$\leftarrow$ if G is a DAG then G has a topo-sort:

Proof: We proved the topo-sort algorithm finds a topo-sort for every DAG. ■